

L&T EduTech
Industrial Internet of Things
Flood Monitoring System
Mid-Course Project Report
Submitted by: Ayan Tandon

GitHub Repository: <https://github.com/YOUR-USERNAME/flood-monitoring-system>

Wokwi Simulation: <YOUR-WOKWI-LINK-HERE>

Demo Video: <YOUR-YOUTUBE-LINK-HERE>

ThingSpeak Dashboard: <YOUR-THINGSPEAK-CHANNEL-LINK-HERE>

1. Aim

To design and simulate an Industrial IoT-based flood monitoring system that continuously measures water levels at a dam using an ultrasonic sensor, provides local alerts through LEDs and a buzzer, and sends real-time notifications to authorities via the cloud when water levels exceed a predefined danger threshold.

2. Problem Statement

Floods are one of the most devastating natural disasters, often caused by rising water levels at dams and reservoirs going undetected until it is too late. Manual monitoring of water levels is unreliable, slow, and cannot provide fast enough warnings during sudden water surges.

There is a need for an automated, low-cost IoT-based system that:

Continuously measures water flow levels at a dam without human intervention.

Provides real-time alerts to authorities via the cloud when flow levels exceed a predefined threshold, indicating potential flooding risks.

3. Scope of the Solution

This project delivers a fully simulated end-to-end flood early warning system. The scope includes:

A single ESP32 microcontroller node with an HC-SR04 ultrasonic sensor representing one monitoring point at a dam, simulated using the Wokwi online simulator.

Three-level threshold-based detection — Safe, Warning, and Danger — with corresponding local LED and buzzer responses.

Cloud-based data logging to ThingSpeak, which plots water level and status code readings on live charts every 16 seconds.

Real-time Discord Webhook notifications sent to an authority monitoring channel when the danger threshold is crossed, and a follow-up all-clear notification when the water level returns to normal.

Full project documentation, circuit simulation, flowchart, and a recorded demo video.

Out of scope: physical hardware deployment at an actual dam site, multi-node sensor networks, rainfall or weather data integration, and long-term predictive analytics. These are natural extensions for a future production-grade version of the system.

4. Required Components

4.1 Hardware (Simulated in Wokwi)

Component	Purpose
ESP32 DevKit V1	Microcontroller with built-in Wi-Fi to read the sensor and connect to the cloud
HC-SR04 Ultrasonic Sensor	Measures distance to the water surface to calculate the dam water level
Green LED	Local visual indicator for Safe state
Yellow LED	Local visual indicator for Warning state
Red LED (blinking)	Local visual indicator for Danger state
Buzzer	Local audible alarm when danger threshold is crossed
330 Ohm Resistors (x3)	Current-limiting resistors for the three LEDs

4.2 Software

Arduino C++ — firmware language used to program the ESP32

Wokwi Online Simulator — used to build, wire, and run the virtual circuit without physical hardware

WiFi, HTTP Client., WiFi Client Secure. — built-in ESP32 Arduino core libraries used for Wi-Fi connectivity and HTTP requests to the cloud (no external libraries required)

4.3 Cloud Environment

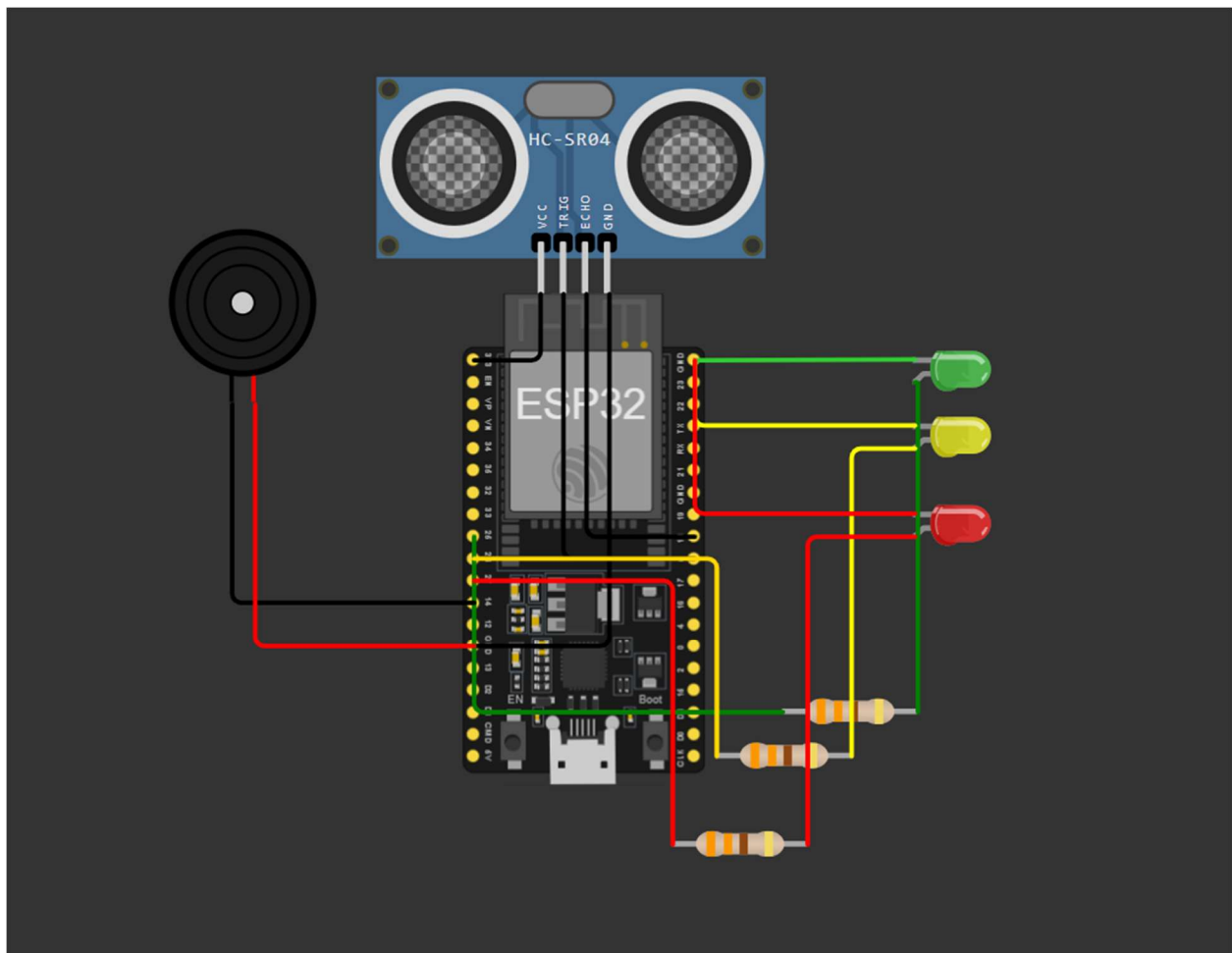
ThingSpeak — free IoT data platform used to log water level readings and status codes, displayed as live charts on a channel dashboard. Field 1 logs water level in cm, Field 2 logs the status as a numeric code (0 = Safe, 1 = Warning, 2 = Danger).

Discord Webhooks — free real-time messaging service used to send instant notifications to an authority monitoring channel when the danger threshold is crossed, and a follow-up all-clear message when the water level returns to normal.

Wokwi-GUEST Network — the simulator's built-in internet-connected Wi-Fi, which allows the simulated ESP32 to reach the real ThingSpeak and Discord cloud services during simulation.

GitHub — used to host and submit the full project including source code, circuit diagram, flowchart, and report.

4.4 Circuit Diagram



5. Working Principle

The HC-SR04 sensor is mounted above the water surface facing downward. It emits an ultrasonic pulse and measures the time taken for the echo to return, which is converted to a distance in centimeters. The water level is then calculated as:

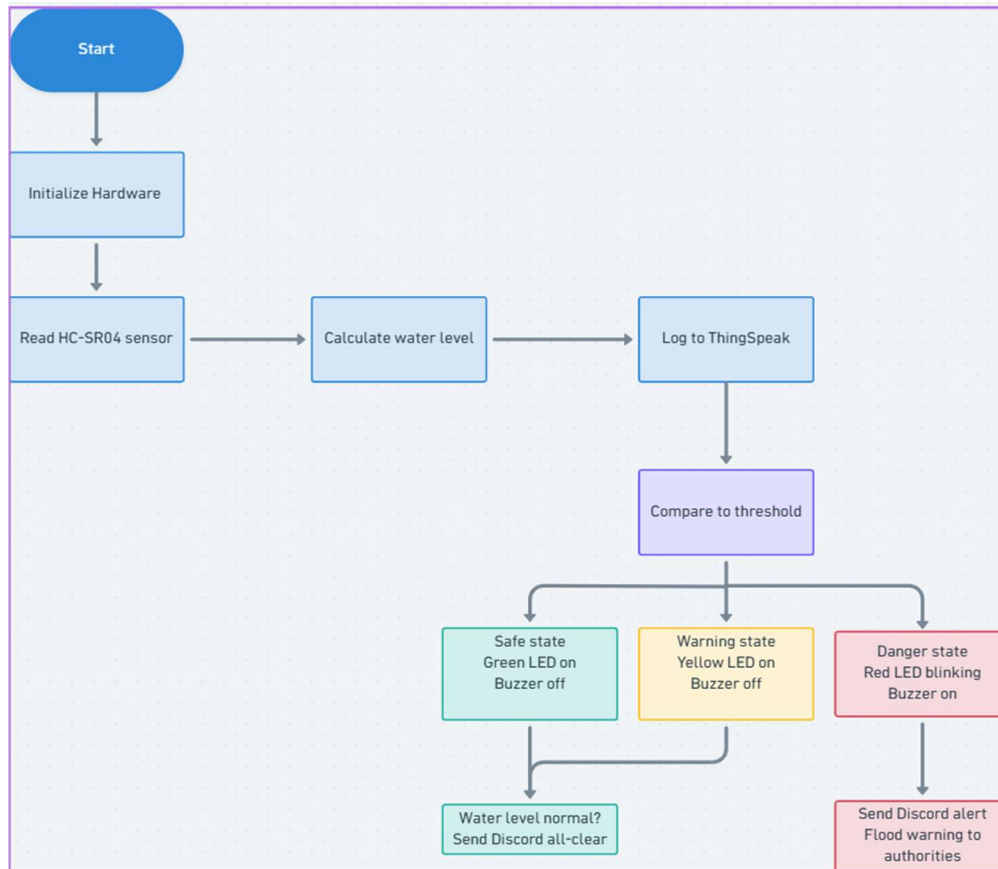
$\text{Water Level (cm)} = \text{Sensor Height} - \text{Measured Distance}$

As water rises, the distance decreases and the water level value increases. Every 2 seconds the ESP32 reads this value and compares it against two thresholds:

Water Level	State	LED	Buzzer	Cloud Action
Below 120 cm	Safe	Green ON	OFF	Data logged to ThingSpeak
120 cm and above	Warning	Yellow ON	OFF	Data logged to ThingSpeak
160 cm and above	Danger	Red BLINKING	ON	Discord flood alert sent
Returns to normal	Safe	Green ON	OFF	Discord all-clear sent

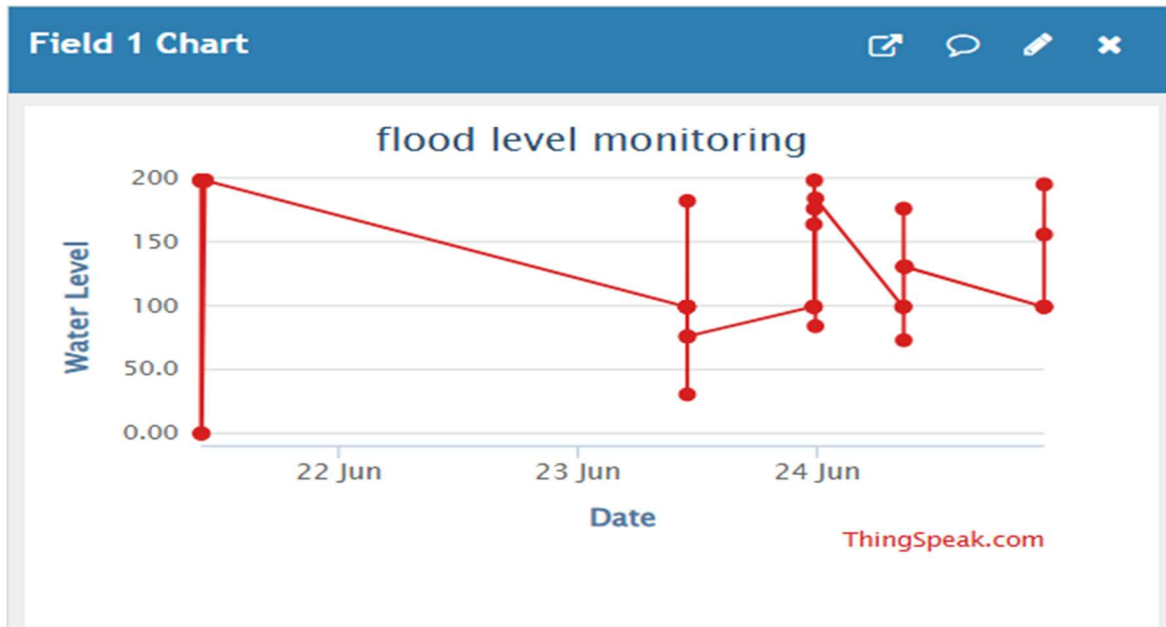
ThingSpeak receives a data update every 16 seconds regardless of state, maintaining a continuous log of the water level over time. The Discord alert is sent only once per danger event to avoid spamming the authorities channel with duplicate notifications.

5.1 Flowchart

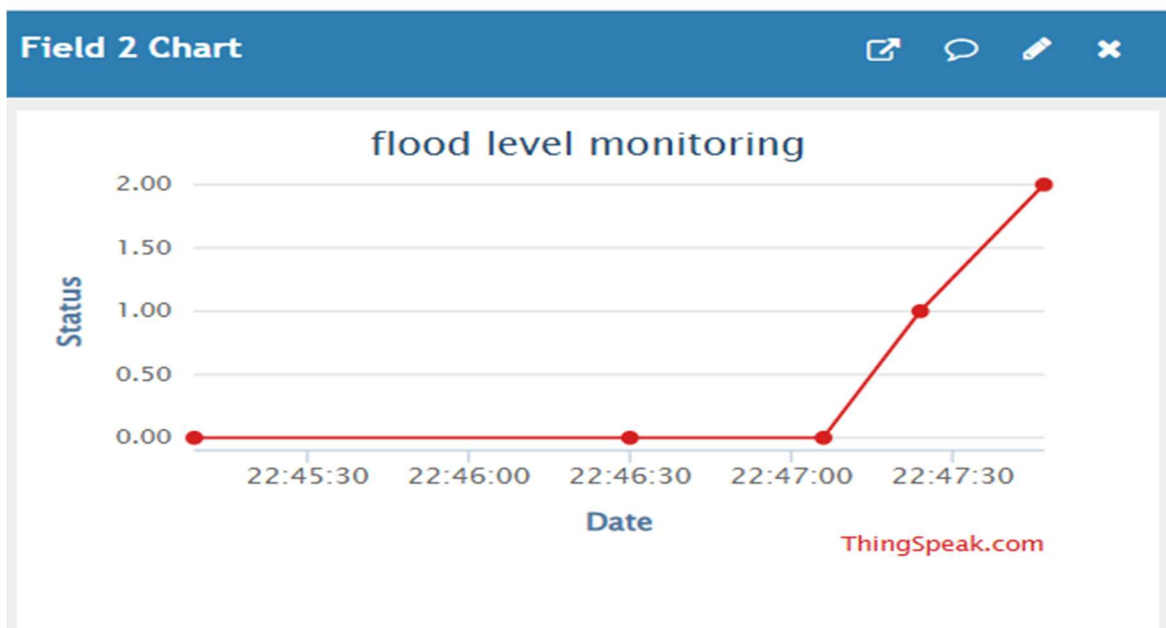


6. Cloud Dashboard Screenshots

6.1 ThingSpeak Data Charts



Field 1 Chart



Field 2 Chart

6.2 Discord Alert Notifications

